

Bias vs Bias - Dawn of Justice for R. Systems

By Michael Antoniadis



Contents

1. Introduction.....	1
2. Problem Overview	7
3. Dataset & Preprocessing.....	9
4. Baseline Recommendation Model	11
5. Fairness-Aware Recommendation Method	13
6. Fairness-Aware Recommendation Method	15
7. Fairness-Aware Recommendation Method	21
8. Runtime Optimization.....	24
9. Robustness and Group Analysis	26
9.1 Active vs Inactive Users	27
9.2 Cold-Start vs Active Users	28
9.3 Intersectional Fairness: Gender x Age	29
10. Top-K and Gamma Sensitivity	32
10.1 Top-K Robustness	33
10.2 Gamma Sensitivity	34
11. Fairness Generalization.....	36
12. Important Trade-Off	41
13. Evaluation	43
14. Conclusion & Future Work.....	45
References	47

Figures	Description
Figure 1	Bias reduction before and after applying fairness-aware re-ranking
Figure 2	Accuracy comparison before and after fairness-aware re-ranking
Figure 3	Fairness-accuracy trade-off for different β values
Figure 4	HitRate changes across different β values
Figure 5	Bias comparison across baseline, fixed fairness re-ranking and adaptive fairness
Figure 6	Accuracy comparison across baseline, fixed fairness re-ranking and adaptive fairness
Figure 7	Runtime comparison between the original fair re-ranking method and the optimized implementation
Figure 8	Bias comparison between active and inactive users
Figure 9	Fairness comparison between cold-start and active users
Figure 10	Intersectional fairness robustness across gender x age groups
Figure 11	Fairness improvement per intersectional group
Figure 12	Fairness and accuracy across different recommendation list lengths
Figure 13	Sensitivity analysis for different γ values
Figure 14	Fairness transfer across age groups
Figure 15	Fairness generalization across occupation groups
Figure 16	Global fairness comparison across demographic attributes
Figure 17	Fairness robustness heatmap across demographic categories
Figure 18	Important trade-off between fairness and accuracy
Figure 19	Final evaluation comparison of baseline, fair paper method, adaptive method and optimized fair method across accuracy, fairness and runtime metrics

1. Introduction

Recommendation systems are used on lots of platforms like Netflix, Spotify and Facebook. They help people find things they like by guessing what they want to watch or listen to.

The problem with these systems is that they are mostly trying to be accurate. They want to show people the things they will like the most. Sometimes they can be unfair to certain groups of people. This means that some people might not get to see many different things or they might see things that are not really meant for them.

In this project we looked at a movie recommendation system to see how this works. We built a system that uses what people have rated before to make new recommendations. Then we checked to see how well it worked by looking at things like how it got it right. If it was fair to everyone.

What we want to do is make sure everyone gets a chance to see things they like without making the recommendations worse. To do this we used a way of re-ranking the recommendations that takes into account both what people like and what is fair.

First we made a list of recommendations based on what people liked. Then we used a method to make sure the list was fair for everyone. We tried two ways of doing this: one where we used a fixed rule to make it fair and another where we looked at each person individually to make sure it was fair for them.

Overall we are trying to figure out how to make recommendation systems that're both accurate and fair. We want to make sure that people get to see things they like. Also that everyone is treated fairly. Recommendation systems like these can be really useful. They need to be fair, to everyone, including movie recommendation systems.

2. Problem Overview

A movie recommendation system is a tool that suggests movies to users based on what they like what they have seen before or what they might be interested in. In this project we are working on a system that recommends movies by figuring out which ones each user will enjoy the most.

However these systems can be unfair. Unfairness happens when the movie recommendations given to users are not equal across groups of people like men and women or young and old. For example one group might only get recommendations for action movies while another group gets recommendations for a variety of movie genres.

It's essential to make sure that movie recommendation systems are fair because they control what users see and discover. If a system keeps suggesting the types of movies to certain groups it can limit their exposure to different kinds of movies and make the recommendations less diverse.

Reducing unfairness is crucial. It can also affect how good the recommendations are. A system that prioritizes fairness might rank some relevant movies lower while a system that prioritizes accuracy might keep suggesting unfair movie recommendations. So in this project we are trying to find a balance between fairness and how good the movie recommendations are.

We want to make sure that our movie recommendation system suggests a variety of movies to all users while still making sure that the recommendations are accurate and relevant. By doing we can create a system that is both fair and effective.

However recommendation systems can contain bias.

Bias appears when the recommendations given to users are not balanced or fair across demographic groups, such as gender or age.

Demographic fairness matters because recommender systems influence what users see, consume and discover.

If a system repeatedly favors patterns, genres or groups it can reinforce unfair exposure and limit diversity in the recommendations.

Reducing bias is important. It can also affect accuracy.

A system that focuses on fairness may move some highly relevant movies lower in the ranking while a system that focuses only on accuracy may keep unfair recommendation patterns.

Therefore this project studies the balance, between fairness and recommendation quality.

Concept	Meaning
Baseline Recommender	The original recommendation model based only on predicted relevance
Fair Re-ranking	Reordering recommendations using both relevance and fairness
Bias	Difference between the recommendation distribution and the desired fair distribution
KL Divergence	Metric used to measure distributional bias
HitRate	Measures whether at least one relevant movie appears in the recommendation list
NDCG	Ranking-quality metric that rewards relevant items appearing near the top

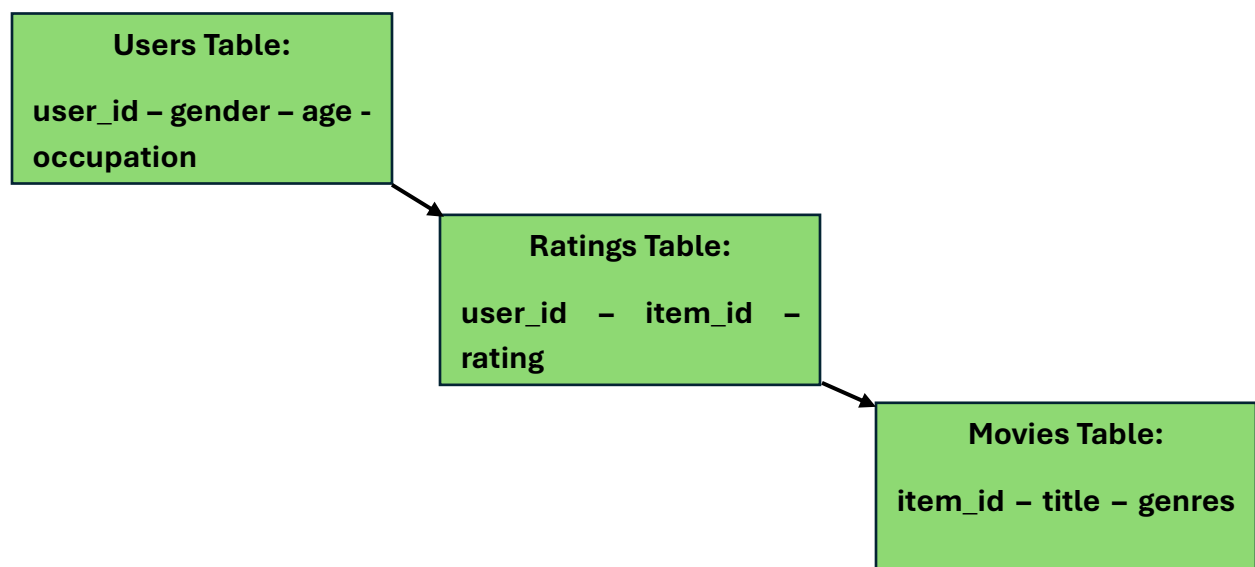
3. Dataset & Preprocessing

The project is using a dataset that's similar to the **MovieLens** dataset. This dataset has information about the users, the movies and the ratings that the users give to the movies. The dataset is divided into three parts: one part for the users one part for the ratings and one part for the movies.

The part for the users has information like the **user_id**, the **gender of the user** the age of the user and what the user does for a living. The part for the ratings has information about how the users interact with the movies using the **user_id**, the **item_id** and the **rating** that the user gives to the movie. The part for the movies has information like the **item_id**, the title of the movie and the type of movie it is.

Before we started building the system that **recommends movies** we did some work on the data to get it ready. We split the ratings into two groups: one group to help us build the model and one group to test the model and see how well it works. Then we made a table where the rows are the users and the columns are the movies. The values in the table are the ratings that the users gave to the movies or what we think the users would like.

We also took the types of movies. Turned them into special codes, where each type of movie is either a 0 or a 1 depending on whether the movie is that type or not. These codes helped us see what types of movies each user likes. We also got the information about the users, like their gender and age so we can see if the system is fair, to everyone.



Schema 1. Dataset structure showing the relationship between users, ratings, and movies used in the recommendation system.

4. Baseline Recommendation Model

The first model we made for this project is the baseline recommendation model. This model is simple. It just recommends movies based on what it thinks the user will like.

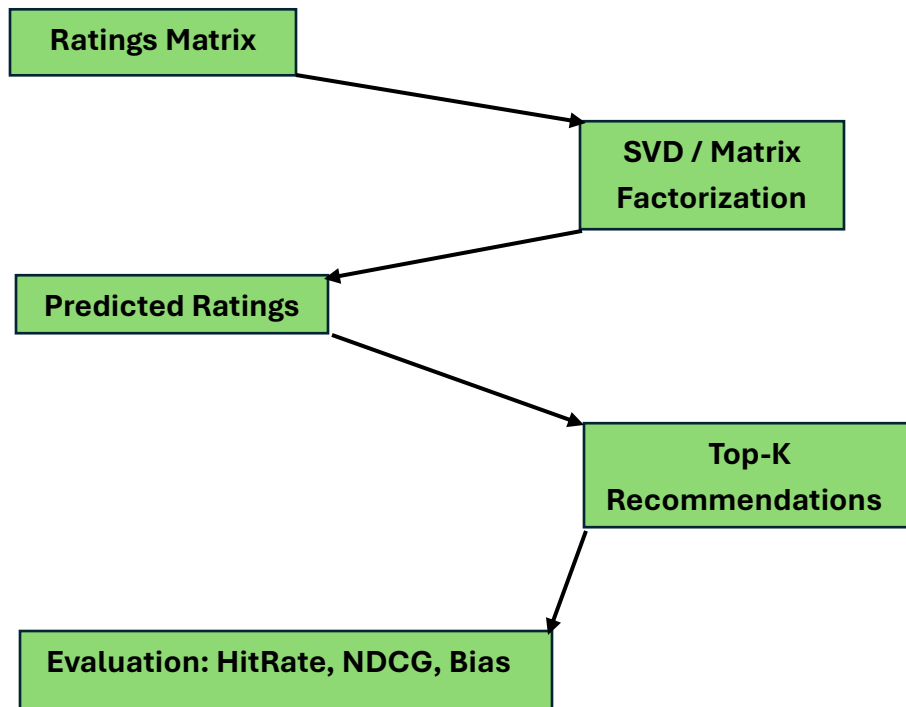
The baseline recommendation model uses something called filtering. We did this by using matrix factorization and **SVD**. What this does is it breaks down the list of user and movie ratings into smaller parts. This helps the model learn things about the users and the movies that are not immediately obvious.

Then the model tries to guess the ratings, for movies that the users have not seen yet. For each user the system makes a list of movies in order of how much the user will like them. The system then recommends the movies to the user.

We used **HitRate** and **NDCG** to see how well the baseline recommendation model works. We also used **KL Divergence** to see if the model is biased in some way. The baseline recommendation model is a part of the project because it helps us understand how the recommendation model works.

Method	HitRate	NDCG	Bias
Baseline	0.957	0.680	0.937

The baseline recommender achieves strong accuracy, with high **HitRate** and **NDCG**. However, it also produces relatively high bias, meaning that the distribution of recommended genres is not equally fair across demographic groups.



Schema 2: Baseline recommendation pipeline using collaborative filtering and matrix factorization before applying fairness-aware re-ranking.

5. Fairness-Aware Recommendation Method

The main part of this project is the fairness- recommendation method.

The system generates recommendations. Then re-ranks them to make sure they are fair and still good for the user.

The method works by comparing two things:

- **The Recommended Category Profile (RCP)**
- **The Counterfactual/Calibration Category Profile (CCP).**

The **RCP** shows what kinds of movies are recommended to a user. For example if recommended movies are Action or Thriller the **RCP** will show that.

The **CCP** is a target to make recommendations fair. It is made using movie preferences of users from a group.

This helps the system make recommendations by comparing them to a balanced target. For each movie the system uses a list of genres. These lists are combined to see what kinds of movies are recommended to each user. Then the system compares this to the fairness target.

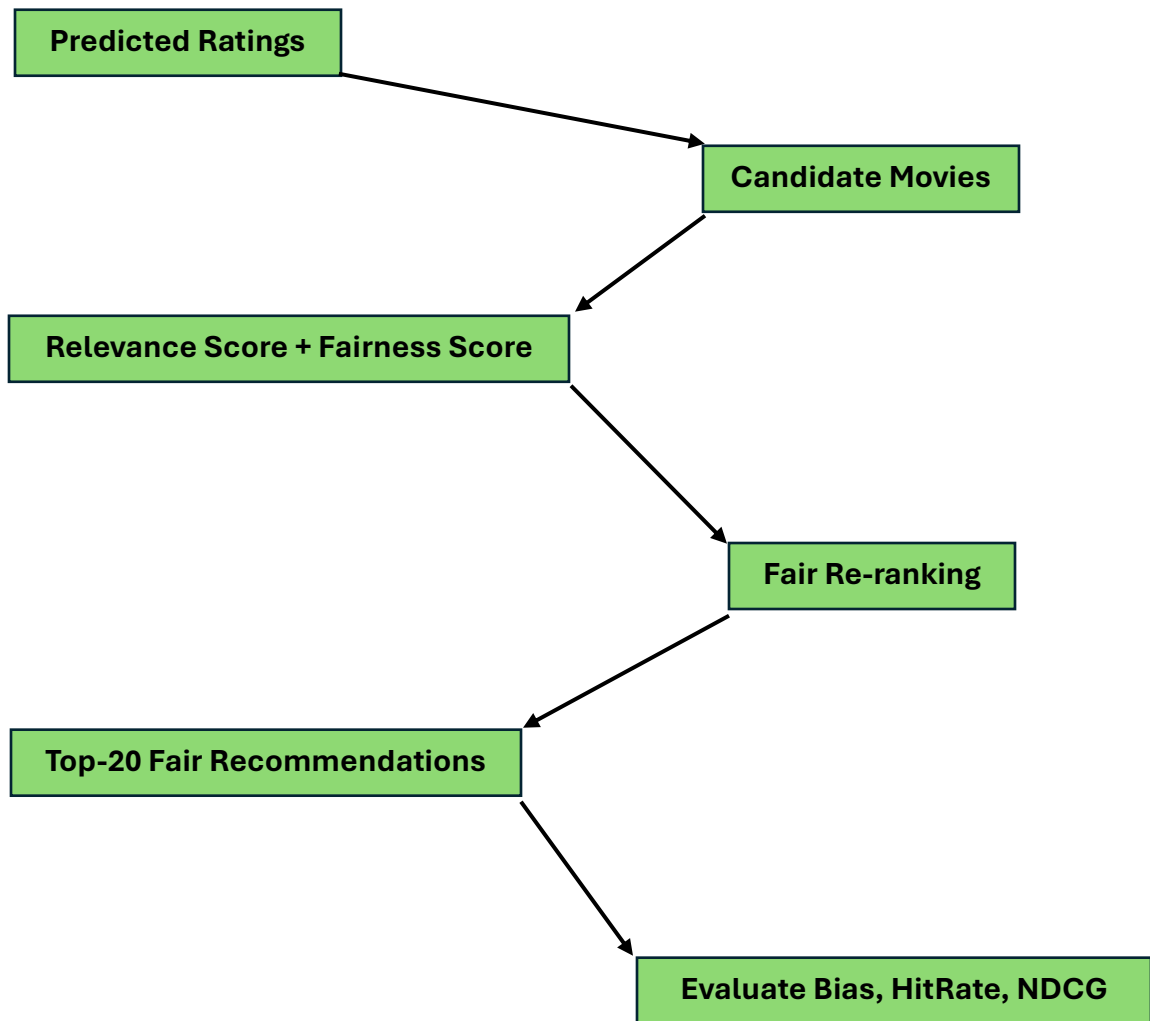
The system uses **KL-divergence** to measure fairness. If KL-divergence is high recommendations are not fair. If it is low recommendations are fair.

To make recommendations the system combines:

- How good a movie is, for the user
- A fairness score.

$$\textit{Final Score} = (1 - \beta) \times \textit{Relevance} + \beta \times \textit{Fairness}$$

The parameter β controls the balance between accuracy and fairness. When $\beta = 0$, the system only uses relevance and behaves like the baseline recommender. When $\beta = 1$, the system gives full priority to fairness. In our experiments, $\beta = 0.5$ gives a balanced trade-off between recommendation quality and fairness improvement.



Schema 3: Fairness-aware re-ranking pipeline. The system combines relevance and fairness scores to produce fairer Top-20 movie recommendations.

6. Fairness-Aware Recommendation Method

This project is really about finding a balance between how accurate the recommendations are how fair they are. The method we use to make the recommendations fairer has a setting called β (beta) that decides how much it cares about being fair compared to showing the most relevant things.

When β is zero the system does not care about fairness at all and just shows what it thinks you will like. As β gets bigger the system starts to care about being fair and tries to make sure that the recommendations are not biased towards certain groups of people. If β gets too big the system might start to care too much about being fair and not enough, about showing you things that are actually relevant.

To see how this works we tried out a different β values:

$$\beta = 0, 0.25, 0.50, 0.75, 0.90$$

The results are shown below:

β	Bias	HitRate	NDCG
0.00	0.937	0.957	0.680
0.25	0.934	0.957	0.680
0.50	0.916	0.955	0.676
0.75	0.921	0.943	0.661
0.90	0.955	0.913	0.575

From these results $\beta = 0.5$ provides the balance between being fair and being accurate. The bias is reduced from **0.937** to **0.916** at this value.

The **HitRate** and **NDCG** remain the same. This means the system becomes fairer without hurting the quality of the recommendations much. On the other hand very high β values make the recommendations worse.

For example when $\beta = 0.9$ the **NDCG** drops to **0.575**. The **HitRate** drops to **0.913**.

This is because the system prioritizes fairness much and moves some really good movies lower in the list.

The **β value** is too high so it hurts the quality of the recommendations. So the goal is to find a β value that's just right. We want to reduce the bias but still keep the personalization. In this project $\beta = 0.5$ was chosen because it is the trade-off point.

The β value of 0.5 reduces the bias. Keeps the quality of the recommendations good.

The β value of 0.5 is the balance, between fairness and accuracy.

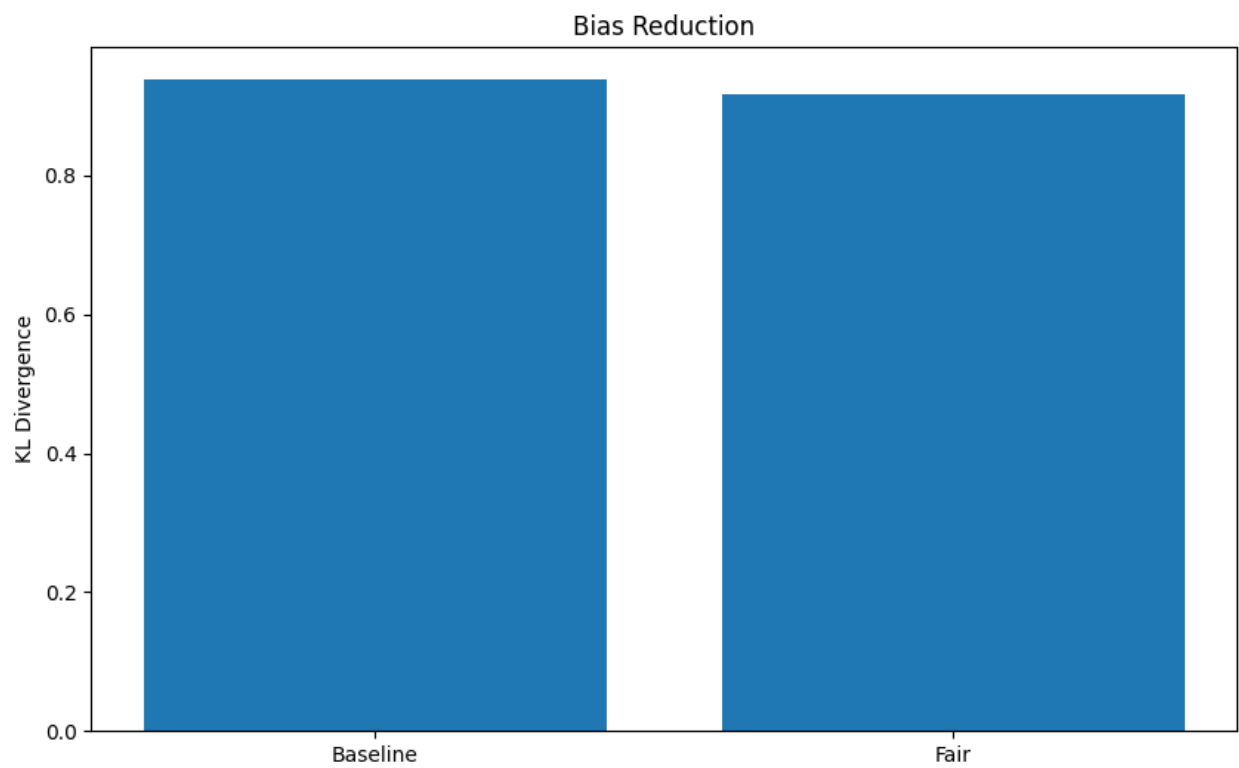


Figure 1: Bias reduction before and after applying fairness-aware re-ranking. The lower KL-divergence after re-ranking shows improved fairness.

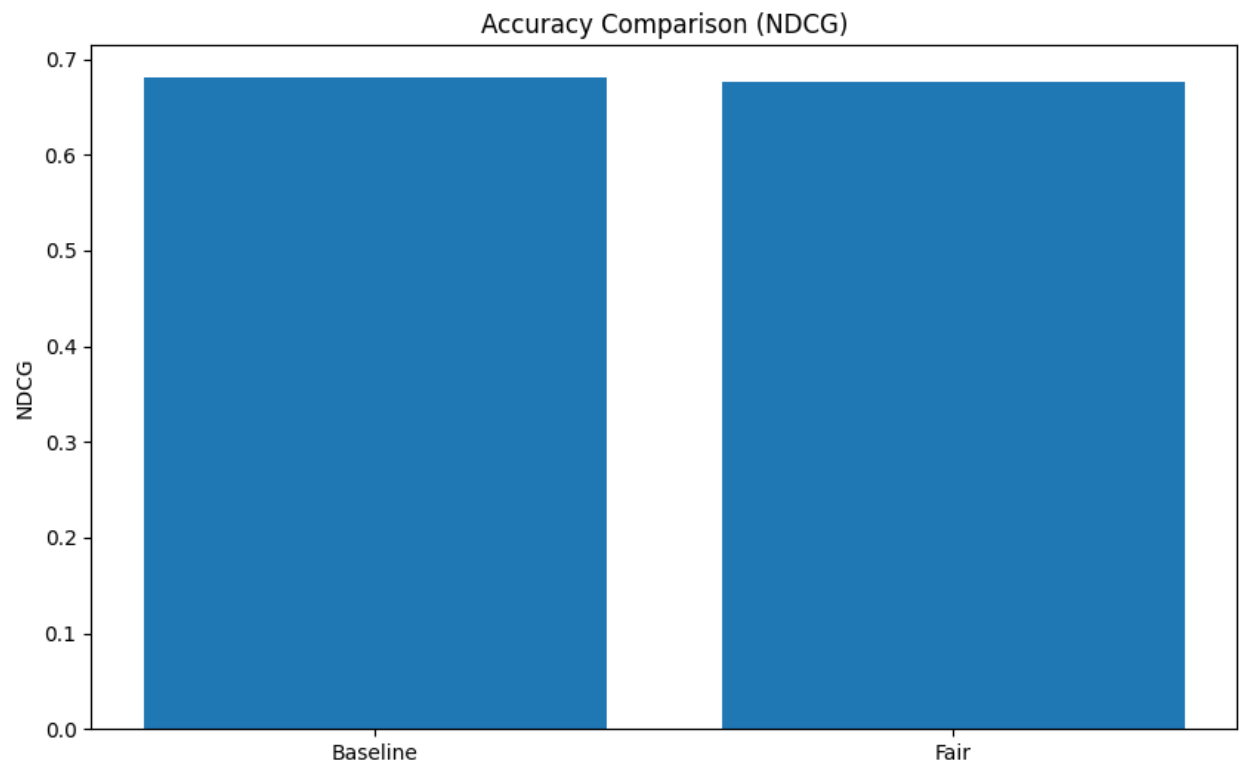


Figure 2: Accuracy comparison before and after fairness-aware re-ranking. The **NDCG** remains almost unchanged, showing that recommendation quality is preserved.

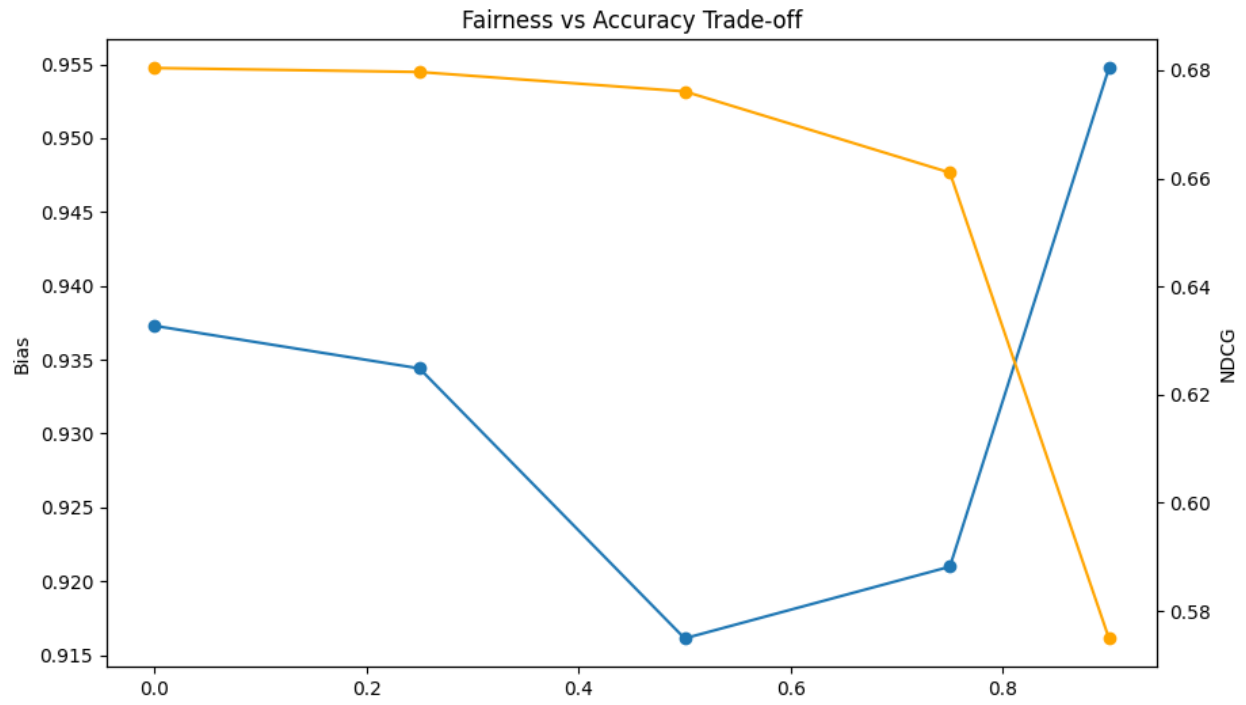


Figure 3: Fairness–accuracy trade-off for different β values. Moderate β values reduce bias while keeping recommendation accuracy high.

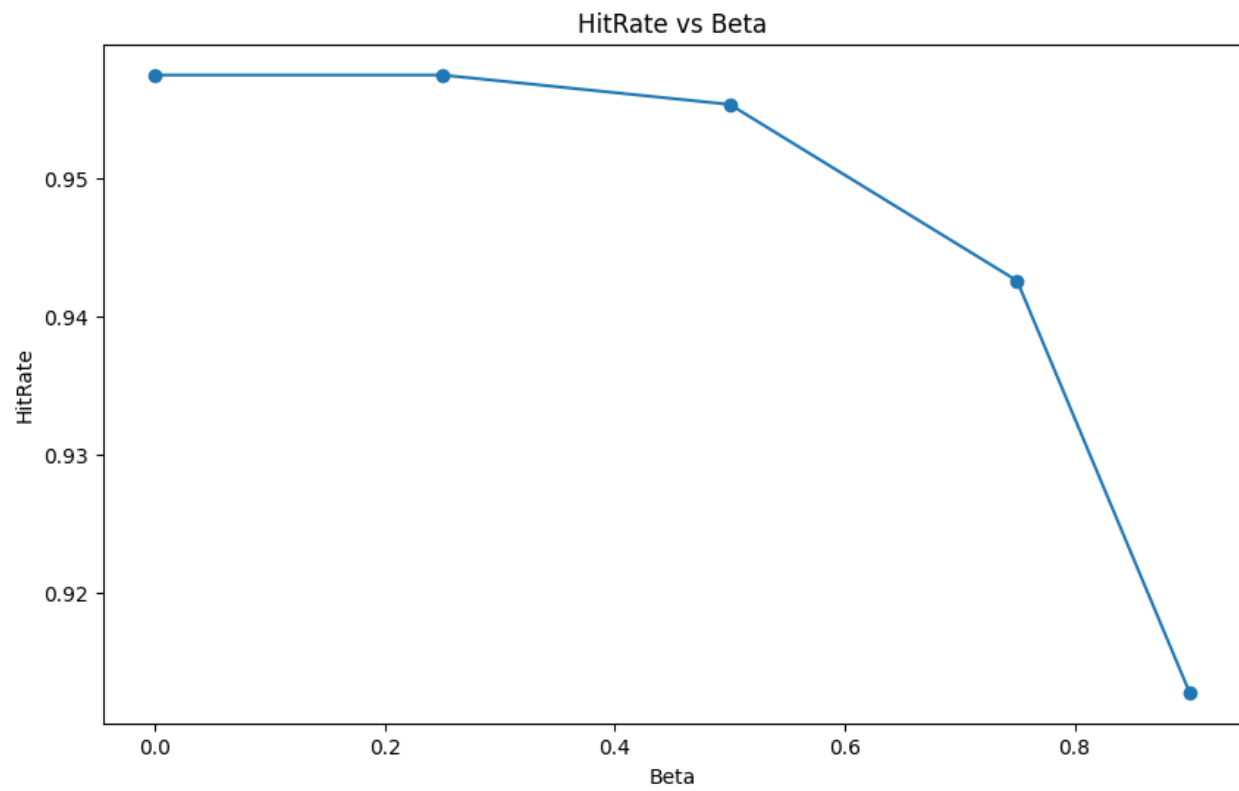


Figure 4: **HitRate** changes across different β values. **HitRate** remains stable for moderate β values but decreases when fairness dominates the ranking process.

7. Fairness-Aware Recommendation Method

In the previous fairness-aware method, the same value of β was used for all users. However, not all users experience the same level of recommendation bias. Some users may already receive relatively balanced recommendations, while others may receive more biased recommendation lists.

For this reason, we implemented an **adaptive fairness method**. Instead of using one fixed β value for every user, the system assigns a personalized fairness weight to each user. Users with higher measured bias receive stronger fairness correction, while users with lower bias are changed less.

The personalized fairness weight is computed from each user's KL-divergence bias score:

$$user_beta = normalized\ user\ KL - divergence$$

This means that users with larger KL-divergence values receive larger β values. The adaptive recommendation score is then computed as:

$$Adaptive\ Score = (1 - \beta_user) \times Relevance + \beta_user \times Fairness$$

This approach allows the system to apply fairness correction only where it is more necessary, instead of modifying all users equally.

The results are shown below:

Method	Bias	HitRate	NDCG
Baseline	0.937	0.957	0.680
Paper Method	0.916	0.955	0.676
Adaptive Method	0.899	0.958	0.680

The adaptive method achieves the lowest bias score, reducing bias from **0.937** in the baseline to **0.899**. At the same time, it preserves almost the same recommendation accuracy as the baseline, with HitRate **0.958** and NDCG **0.680**.

This is one of the strongest results of the project because it shows that fairness can be improved further without sacrificing recommendation quality. The adaptive method is more flexible than the fixed- β method because it adjusts fairness correction according to each user's individual bias level.

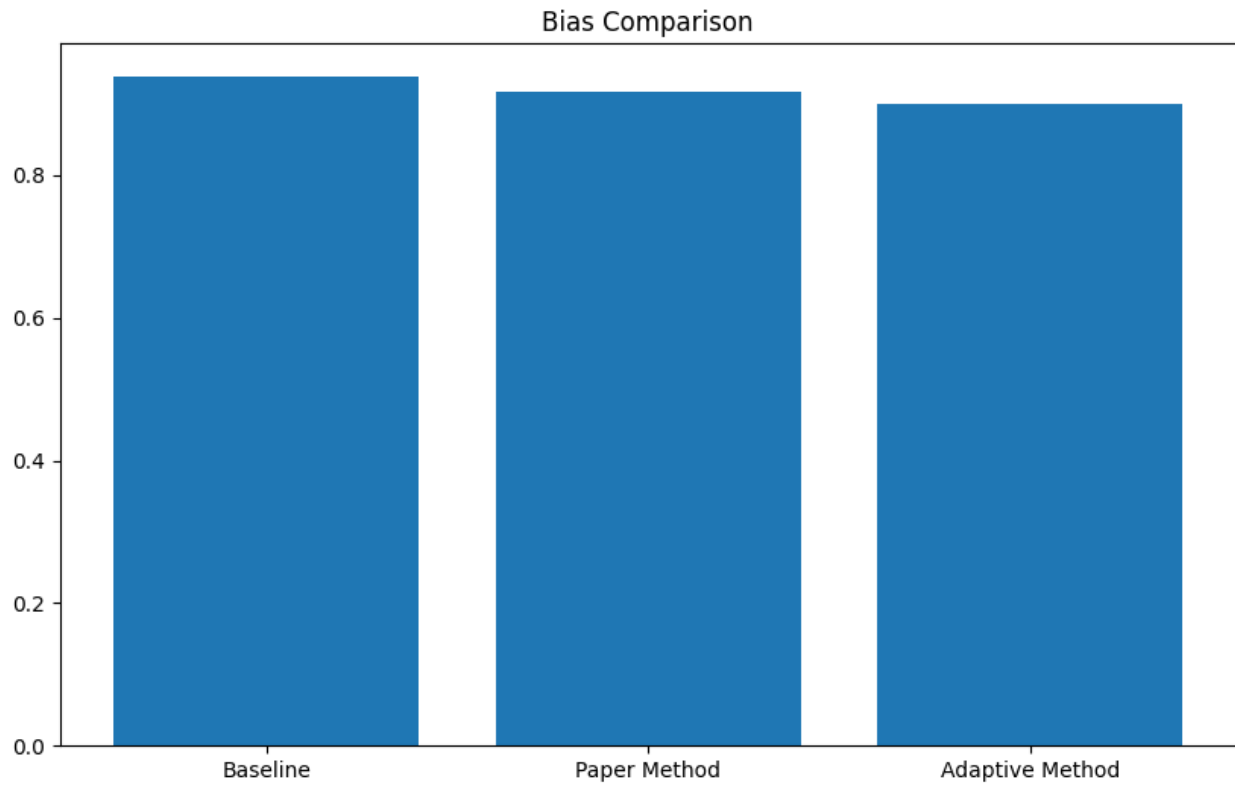


Figure 5: Bias comparison across baseline, fixed fairness re-ranking, and adaptive fairness. The adaptive method achieves the lowest KL-divergence bias.

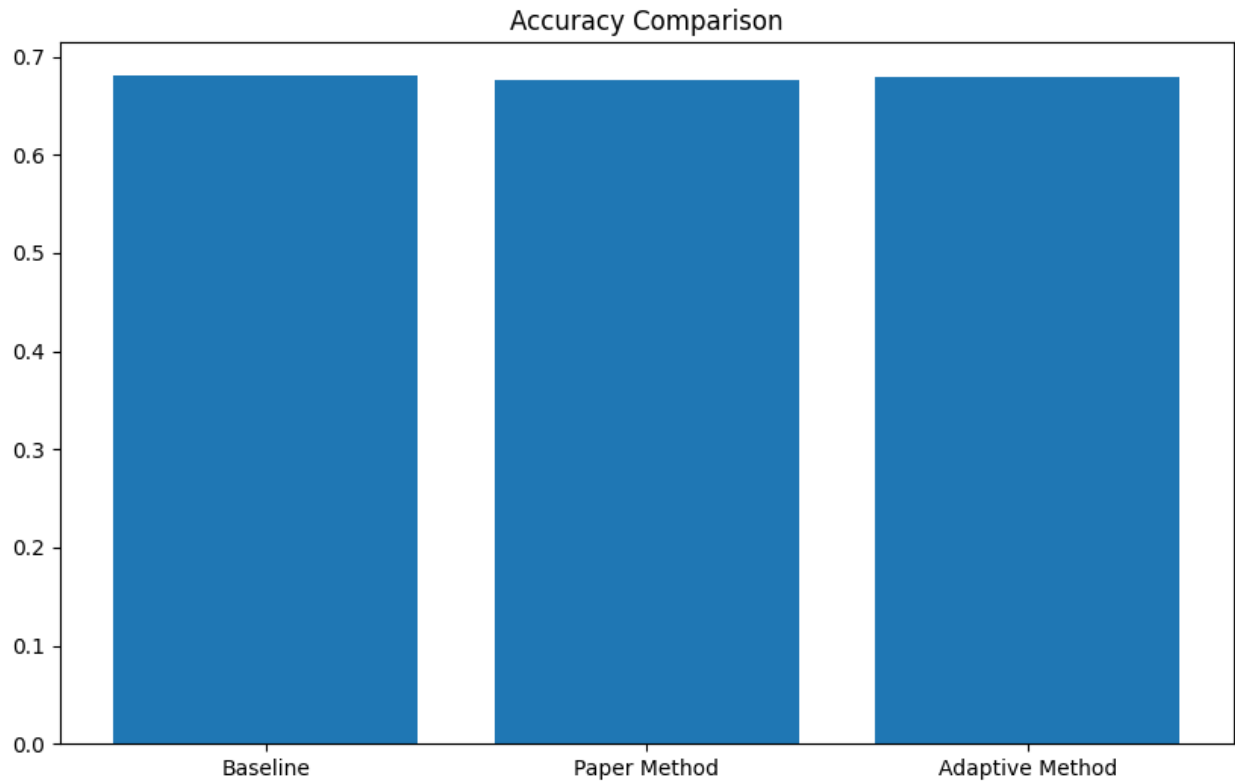


Figure 6: Accuracy comparison across baseline, fixed fairness re-ranking, and adaptive fairness. The adaptive method preserves recommendation quality while reducing bias.

8. Runtime Optimization

Runtime optimization is an important part of this project because the **original fairness-aware re-ranking method** is computationally expensive. In the original implementation, the algorithm loops through users and candidate movies and repeatedly computes genre

information and fairness scores. This makes the method slower when the number of users and movies increases.

To improve efficiency, we implemented an optimized version of the fair re-ranking method. The optimized method precomputes the movie genre matrix and the fairness target vectors before generating recommendations. Instead of recalculating the same values many times, the algorithm reuses these precomputed structures.

The optimized implementation also uses vectorized NumPy operations. This allows many calculations to be performed at once instead of using slow nested loops. As a result, the fair re-ranking process becomes much faster while keeping the same recommendation output.

The runtime results are shown below:

Method	Runtime
Original Fair Method	About 67.94 seconds
Optimized Fair Method	About 1.24 seconds

The optimized method produces the same **HitRate**, **NDCG**, and bias as the original fair method, but runs dramatically faster. This shows that the fairness-aware method can be made much more practical without changing its behavior or reducing its quality.

The final comparison is shown below:

Method	HitRate	NDCG	Bias	Runtime
Baseline	0.957	0.680	0.937	~0
Fair Paper	0.955	0.676	0.916	67.94 s
Optimized Fair	0.955	0.676	0.916	1.24 s

Overall, the optimized fair method keeps the same fairness and accuracy results as the original fair method, but reduces the runtime from about **67.94 seconds** to about **1.24 seconds**. This is one of the main technical improvements of the implementation.

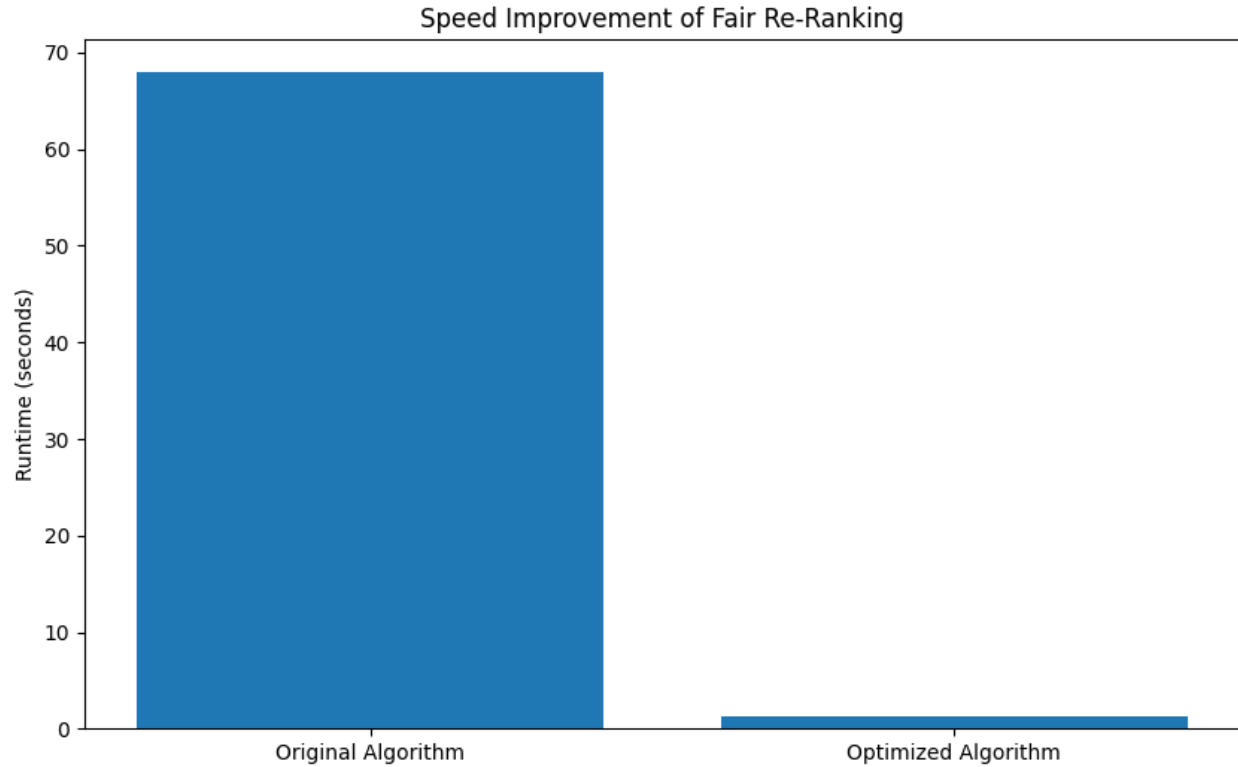


Figure 7: Runtime comparison between the original fair re-ranking method and the optimized implementation. The optimized method is much faster while preserving the same fairness and accuracy results.

9. Robustness and Group Analysis

In addition to the main fairness and accuracy evaluation, we also performed robustness experiments to examine whether the fairness-aware method works consistently across

different types of users. These experiments are important because a method may reduce average bias overall, but still fail for specific user groups.

For this reason, we analyzed fairness across user activity levels, cold-start users, and intersectional demographic groups. All figures in this section should come from **our notebook**, because they represent our own extra experiments and not only the original paper results.

9.1 Active vs Inactive Users

First, users were split based on their activity level. Active users are users with many ratings, while inactive users are users with fewer ratings. The goal of this experiment is to check whether the fairness-aware method behaves differently depending on how much interaction data is available for each user.

After applying the fairness-aware method, bias was computed separately for active and inactive users:

User Group	Bias
Active Users	0.903
Inactive Users	0.894

The results show that the bias values are very close for both groups. This suggests that the fairness-aware method does not only improve fairness for one type of user, but remains relatively stable across different activity levels.

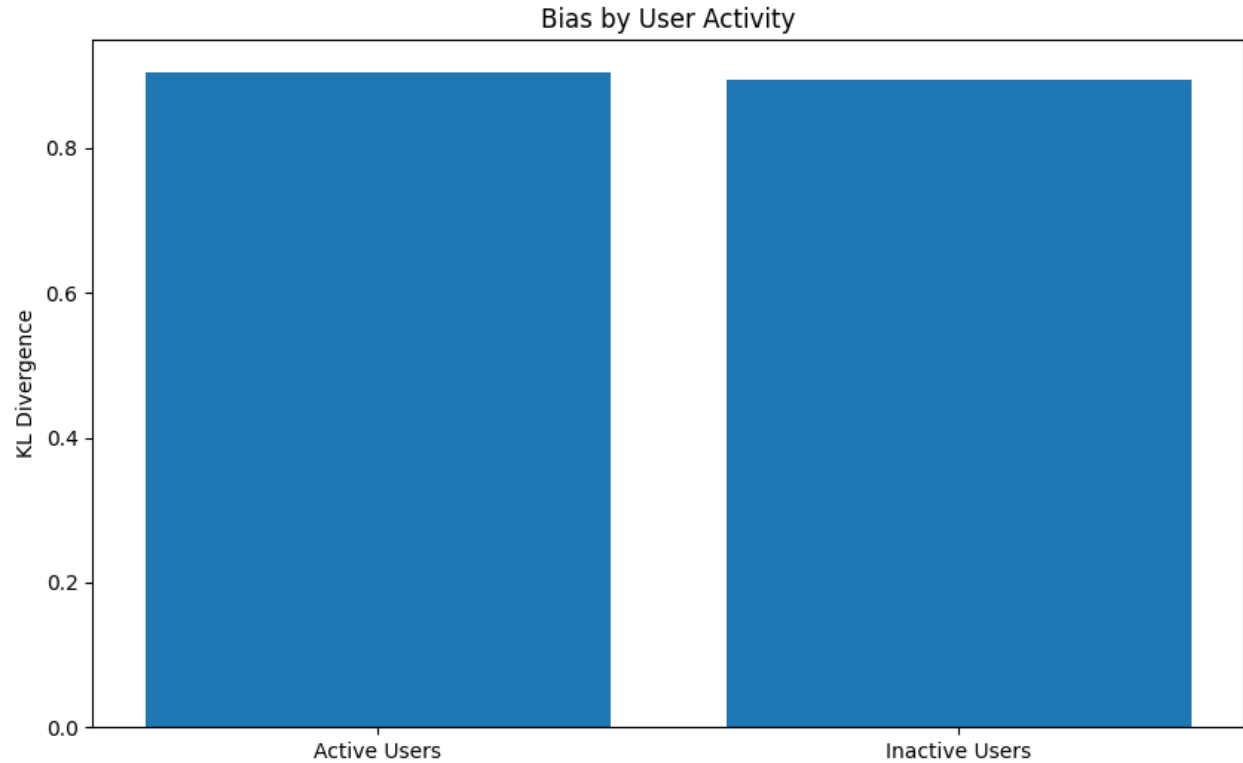


Figure 8: Bias comparison between active and inactive users. The similar KL-divergence values show that fairness remains stable across different user activity levels.

9.2 Cold-Start vs Active Users

The second robustness experiment compares cold-start users with active users. Cold-start users are users with very few interactions, while active users have more rating history

available. This experiment tests whether the fairness-aware method still works when the system has limited information about a user.

The results were:

User Group	Bias
Cold Users	0.939
Active Users	0.893

Cold users still have higher bias than active users. This is expected because the recommender system has less information about their preferences, making it harder to produce both accurate and fair recommendations. This result highlights an important limitation of the method and can be used as a future-work direction.

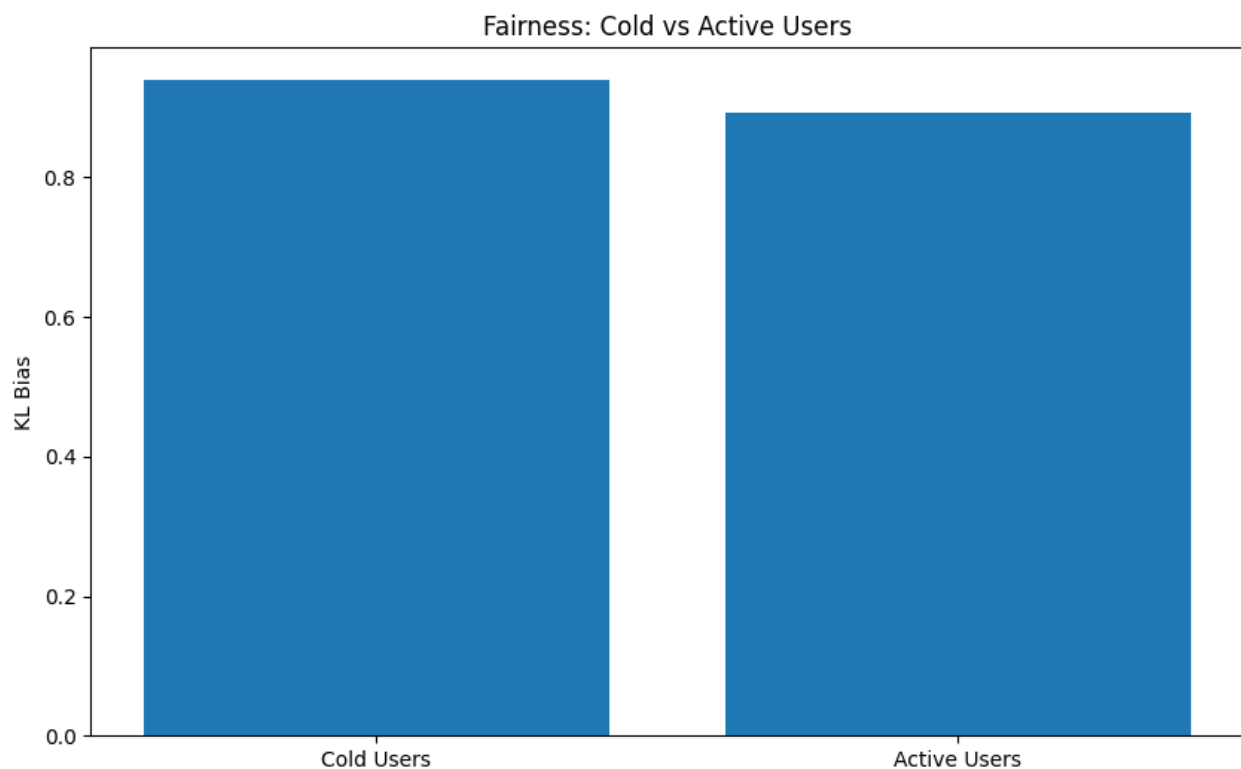


Figure 9: Fairness comparison between cold-start and active users. Cold users show higher bias, indicating that limited interaction history makes fair recommendation more difficult.

9.3 Intersectional Fairness: Gender x Age

The final group-analysis experiment studies intersectional fairness. Instead of evaluating fairness only by gender or only by age, users are grouped using both attributes together. This

creates more detailed demographic groups, such as young female users, middle-aged male users, and older users.

The groups used in this experiment are:

F_young, F_middle, F_old, M_young, M_middle, M_old

This experiment checks whether fairness improvements remain consistent across more specific demographic subgroups. This is important because a recommender system may appear fair on average, but still produce unfair results for smaller intersectional groups.

The results show that the fair method reduces bias for all gender-age groups. However, the amount of improvement is not exactly the same for every group. Some groups benefit more from fairness-aware re-ranking than others, which shows why intersectional evaluation is important.

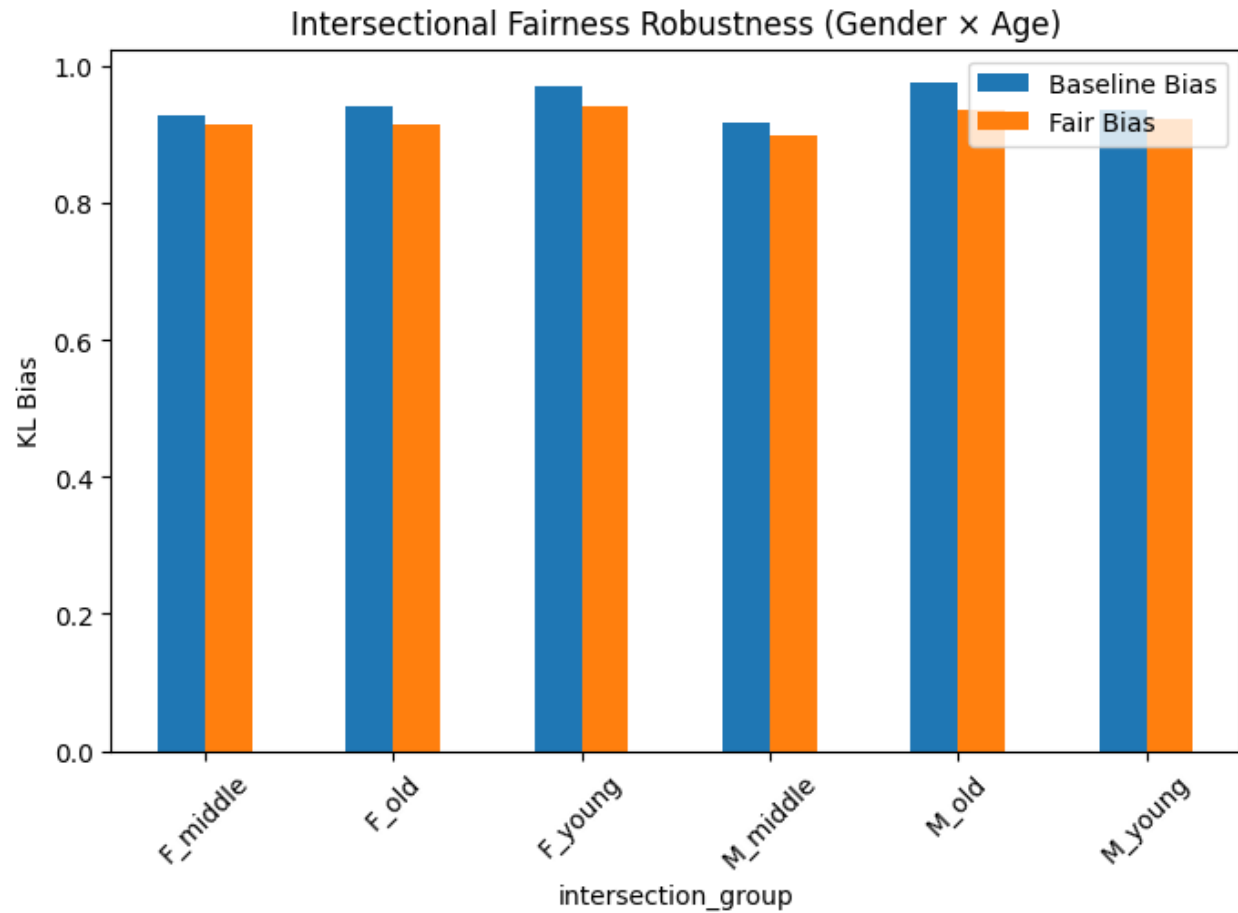


Figure 10: Intersectional fairness robustness across gender $\times$ age groups. The fair method reduces bias across all intersectional groups compared with the baseline.

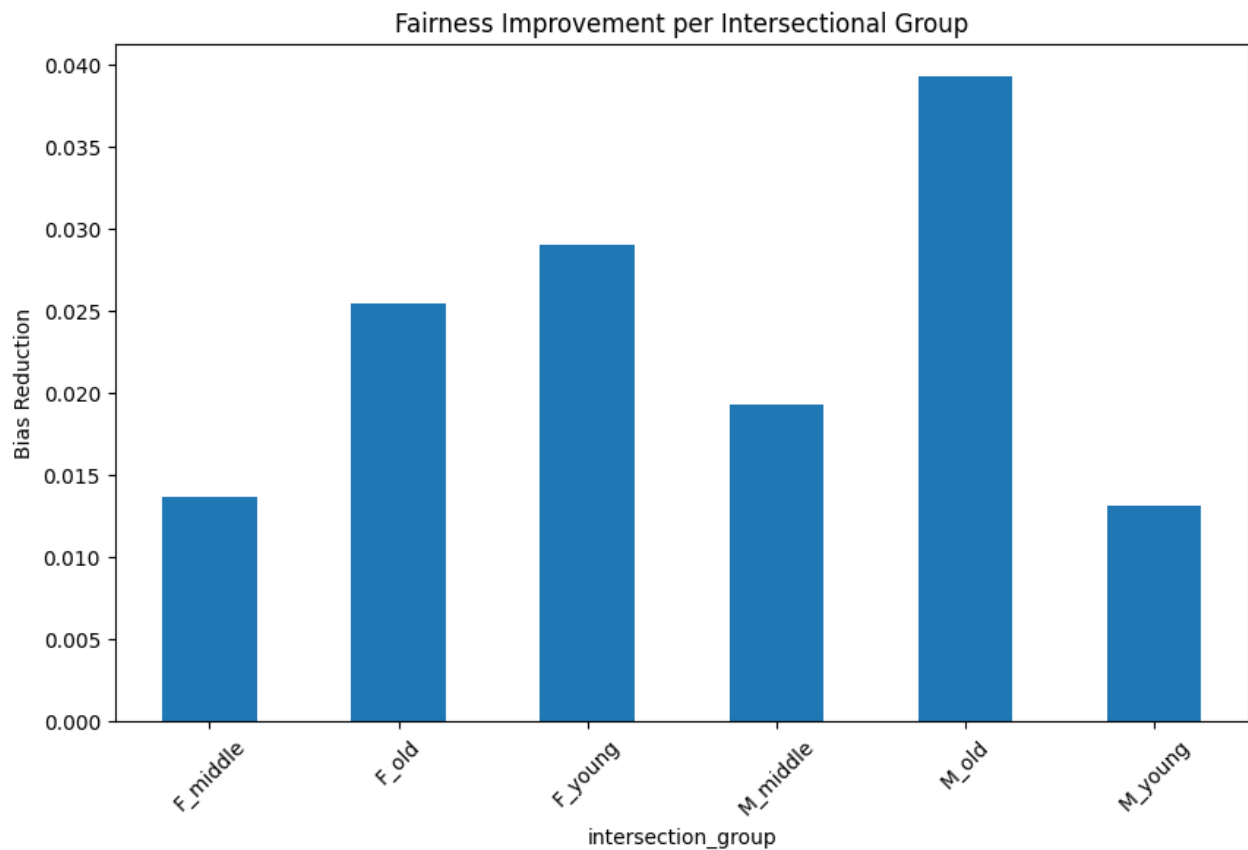


Figure 11: Fairness improvement per intersectional group. The plot shows which gender-age groups benefit most from fairness-aware re-ranking.

10. Top-K and Gamma Sensitivity

To further evaluate the robustness of the fairness-aware recommendation method, we tested how the results change under different recommendation settings. Specifically, we examined two parameters: the recommendation list length **K** and the ranking discount parameter **γ (gamma)**.

These experiments help show whether the method remains stable when the recommendation setup changes. All figures in this section should come from **our notebook**, because they represent our own robustness experiments.

10.1 Top-K Robustness

In the Top-K robustness experiment, we tested different recommendation list sizes:

$$K = 5, 10, 20, 50$$

The purpose of this experiment is to check whether fairness and accuracy remain stable when the system recommends fewer or more movies to each user. A smaller K means the system only evaluates the very top recommendations, while a larger K gives the system more space to include both relevant and fair items.

The results show that as **K increases**, bias decreases and NDCG improves. This means that longer recommendation lists provide more room for the re-ranking method to balance fairness and accuracy. When the list is very short, the system has fewer opportunities to correct bias without affecting relevance.

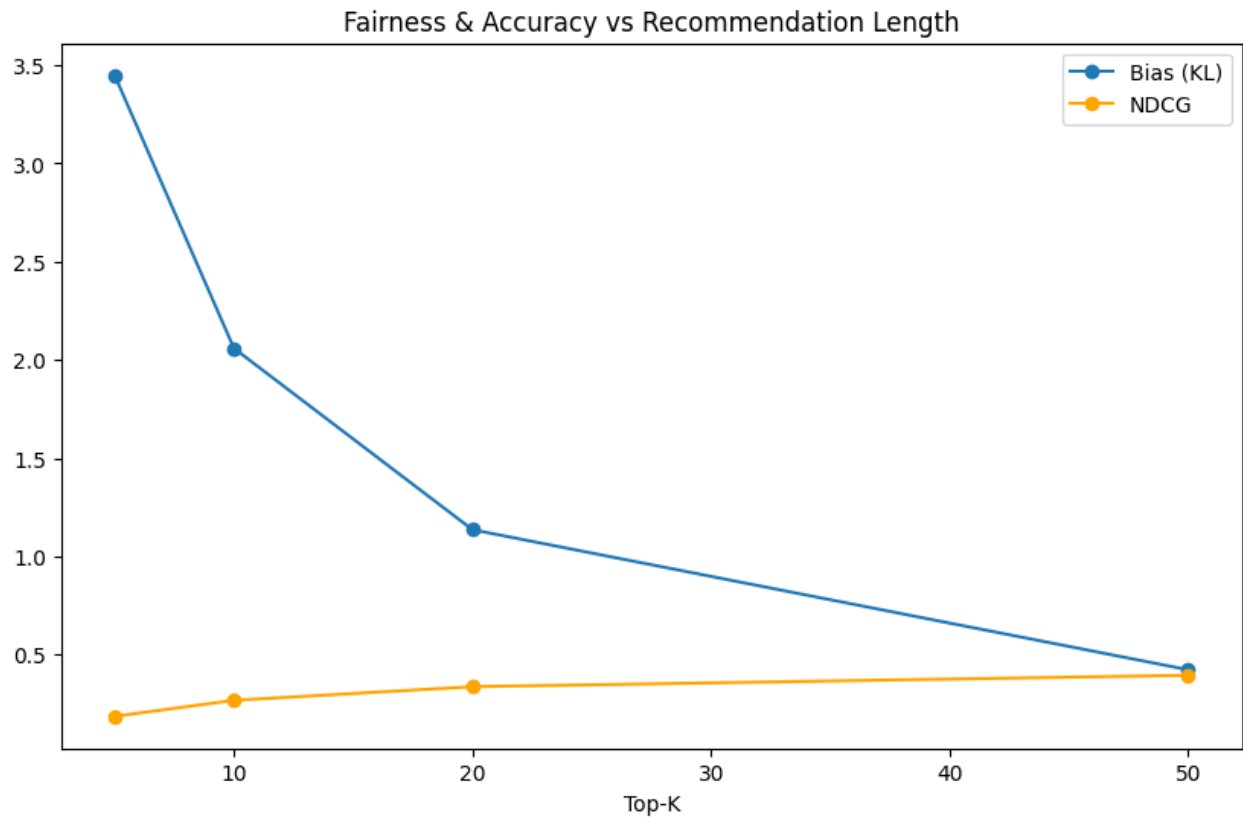


Figure 12: Fairness and accuracy across different recommendation list lengths. As K increases, KL-divergence decreases and NDCG improves, showing that longer recommendation lists allow better fairness–accuracy balance.

10.2 Gamma Sensitivity

The second robustness experiment tests the effect of the ranking discount parameter γ (**gamma**). Gamma controls how strongly the position of an item in the recommendation list affects the fairness calculation.

We tested the following gamma values:

$$\gamma = 0, 0.1, 0.3, 0.5, 1.0$$

The goal of this experiment is to check whether the fairness-aware method is sensitive to changes in the ranking discount. If small changes in gamma caused large changes in bias or accuracy, the method would be unstable.

The results show that both bias and NDCG remain relatively stable across different gamma values. This suggests that the method is not highly sensitive to this parameter and can maintain similar fairness and accuracy under different ranking-discount settings.

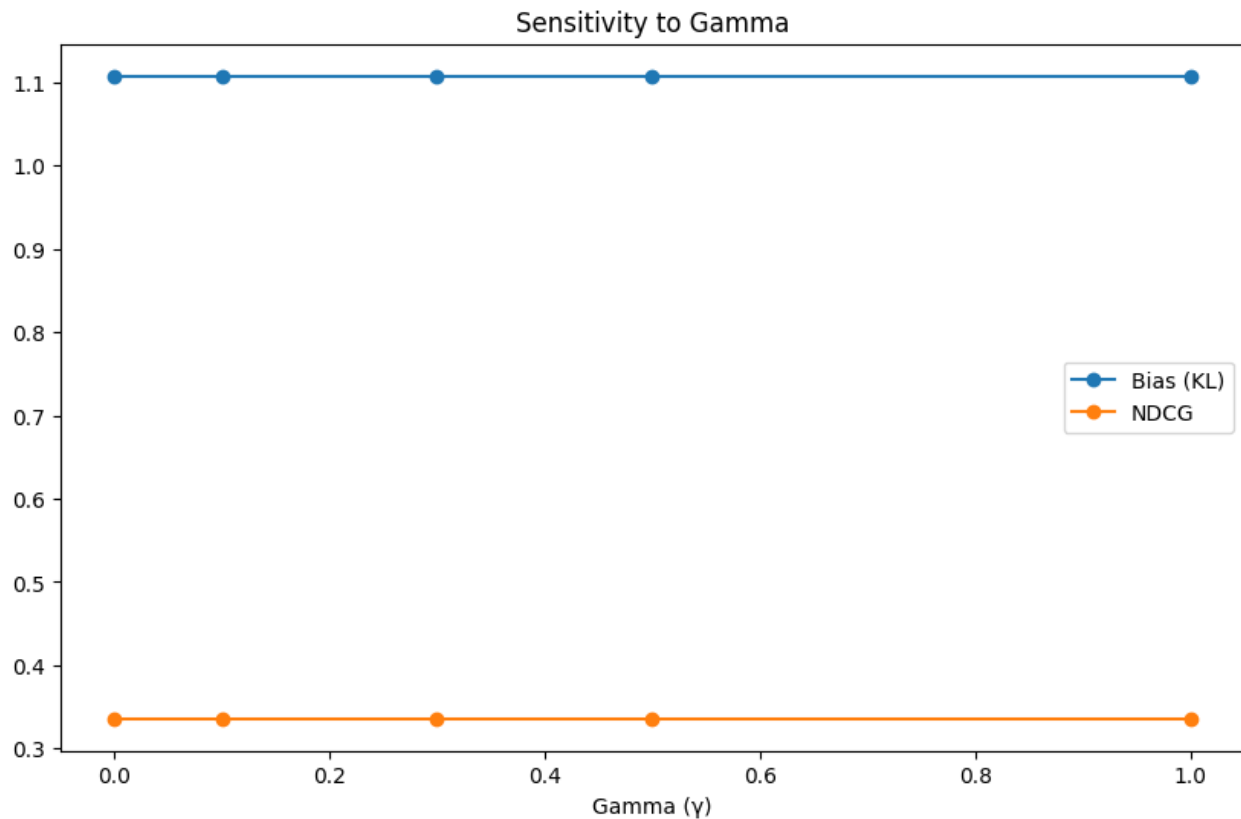


Figure 13: Sensitivity analysis for different γ values. Bias and NDCG remain stable, showing that the fairness-aware method is robust to changes in ranking discount settings.

11. Fairness Generalization

The final experiment of this project studies whether the fairness-aware method generalizes beyond the demographic attribute used during optimization. In our implementation, the fairness method is mainly designed around **gender**, meaning that the re-ranking process uses gender-based fairness targets to reduce recommendation bias.

However, a recommender system should not only appear fair for one attribute. It is also important to check whether fairness improvements transfer to other demographic groups, such as **age** and **occupation**. This experiment therefore evaluates whether reducing gender-related bias also leads to more balanced recommendation behavior across other user attributes.

To test this, we computed bias separately for different age groups and occupation groups after applying the fairness-aware method. This allows us to examine whether the method remains stable across demographic categories that were not the main focus of the fairness optimization.

The results show that the fairness-aware method generalizes reasonably well beyond gender. Bias values remain relatively balanced across age groups and occupation groups, although some differences still exist. This means that the method improves fairness overall, but it does not completely eliminate all subgroup-level disparities.

This experiment is important because it connects directly to the idea of fairness robustness. A method may reduce bias for one protected attribute, but still produce unfair results for other demographic groups. By evaluating gender, age, and occupation together, we obtain a more complete understanding of the fairness behavior of the recommender system.

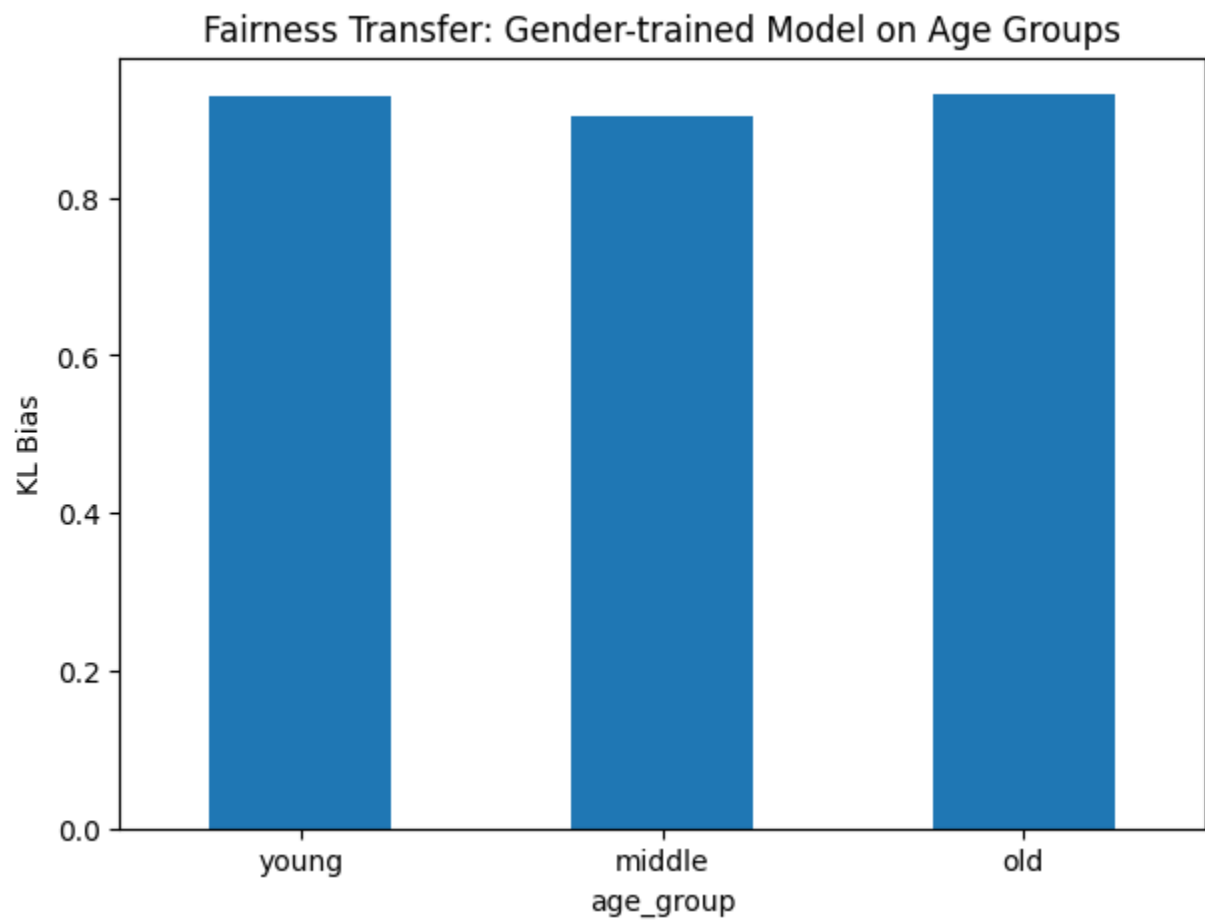


Figure 14: Fairness transfer across age groups. The plot shows whether gender-based fairness re-ranking also produces balanced bias for young, middle-aged, and older users.

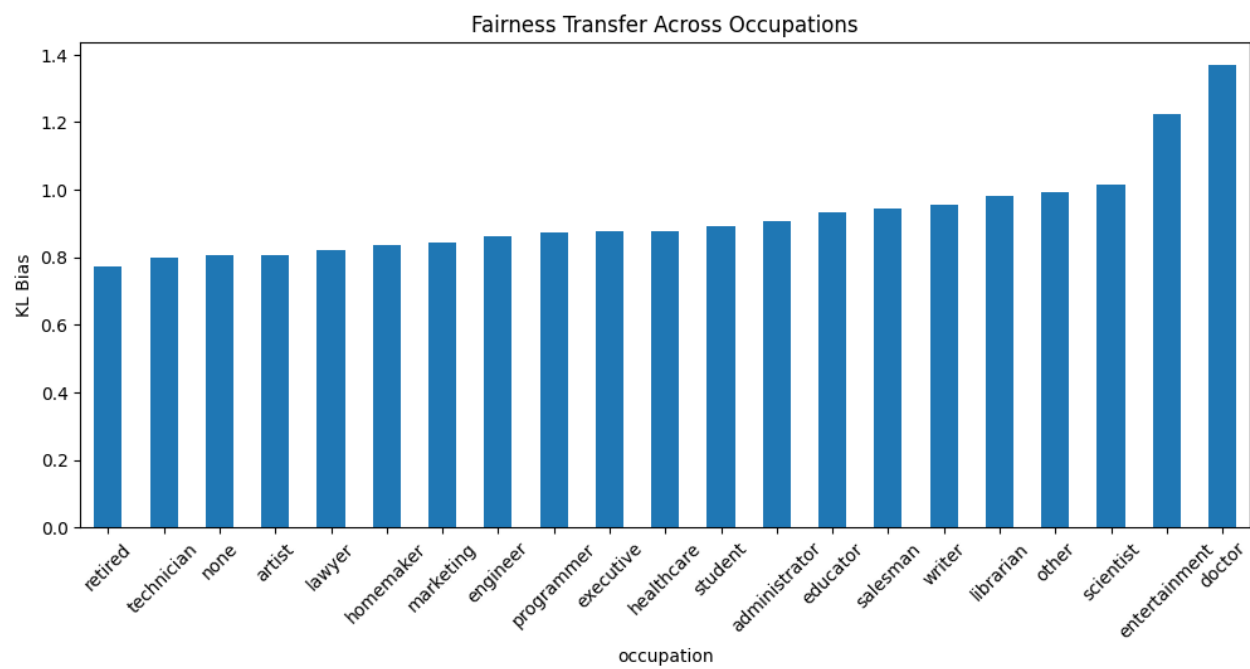


Figure 15: Fairness generalization across occupation groups. The plot shows whether fairness behavior remains stable for users with different occupations.

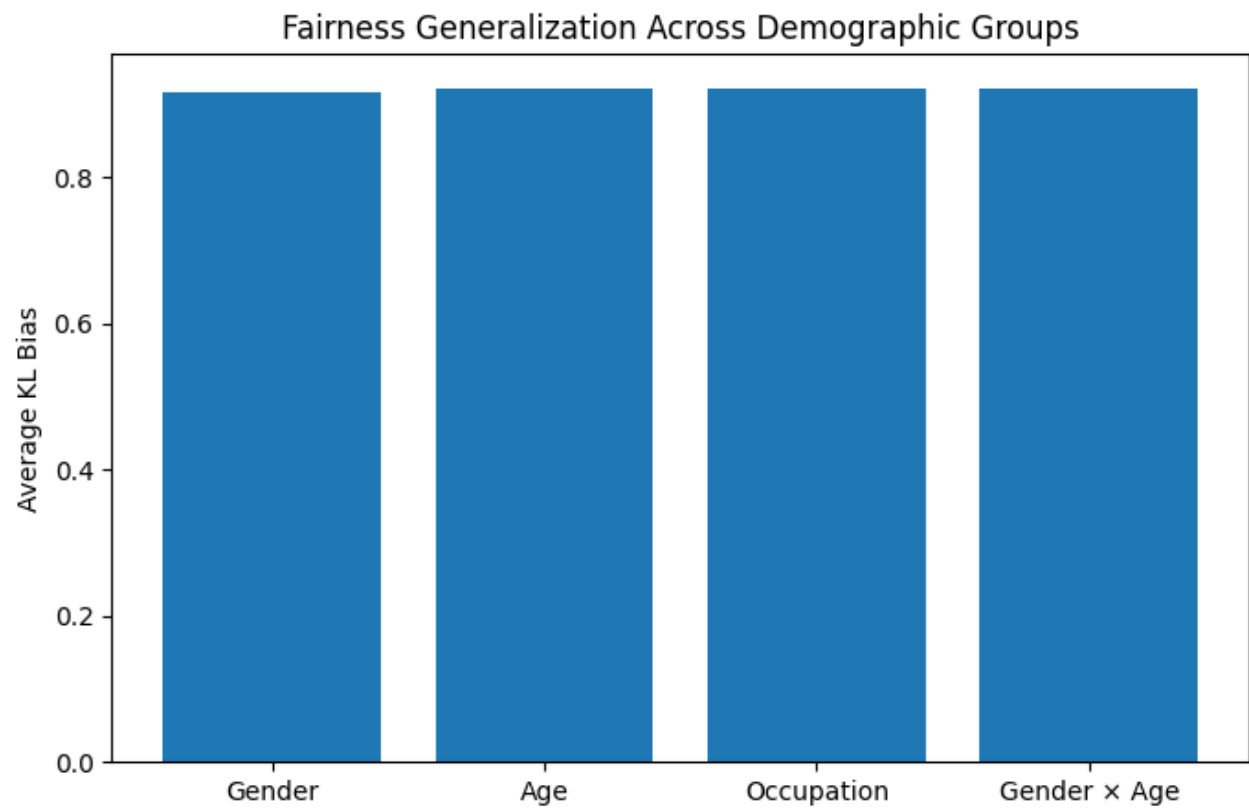


Figure 16: Global fairness comparison across demographic attributes. This figure compares average KL-divergence bias across gender, age, and occupation groups.

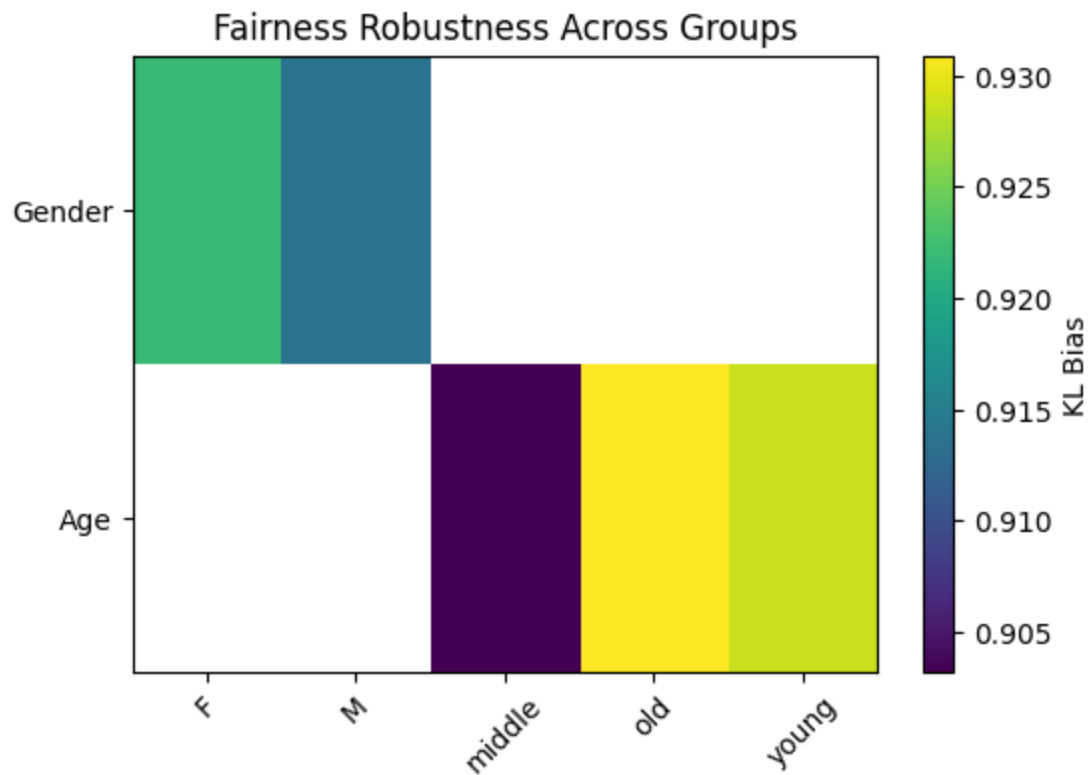


Figure 17: Fairness robustness heatmap across demographic categories. The heatmap provides a compact visual summary of how fairness behaves across multiple user groups.

12. Important Trade-Off

An important trade-off in this project is the balance between **fairness** and **recommendation accuracy**. The goal of the fairness-aware method is to reduce demographic bias in the recommendation lists, but this must be done carefully so that the system does not lose its ability to recommend relevant movies.

The fairness parameter β controls this balance. When β is small, the system focuses mostly on relevance, so recommendation accuracy remains high but bias may remain stronger. When β is large, the system gives more importance to fairness, but this can reduce accuracy because some highly relevant movies may be moved lower in the ranking.

In our experiments, moderate fairness re-ranking improved demographic balance while preserving recommendation quality. Specifically, $\beta = 0.5$ reduced bias from **0.937** to **0.916**, while HitRate and NDCG remained almost unchanged. This shows that fairness can be improved without seriously damaging recommendation performance.

However, when the fairness parameter became too large, recommendation quality decreased. For example, at $\beta = 0.9$, HitRate dropped to **0.913** and NDCG dropped to **0.575**. This means that the system started prioritizing fairness too strongly over relevance.

Therefore, the best solution is not to maximize fairness weight, but to choose a balanced value. In this project, $\beta = 0.5$ was the most effective trade-off because it reduced bias while keeping recommendation accuracy close to the baseline.

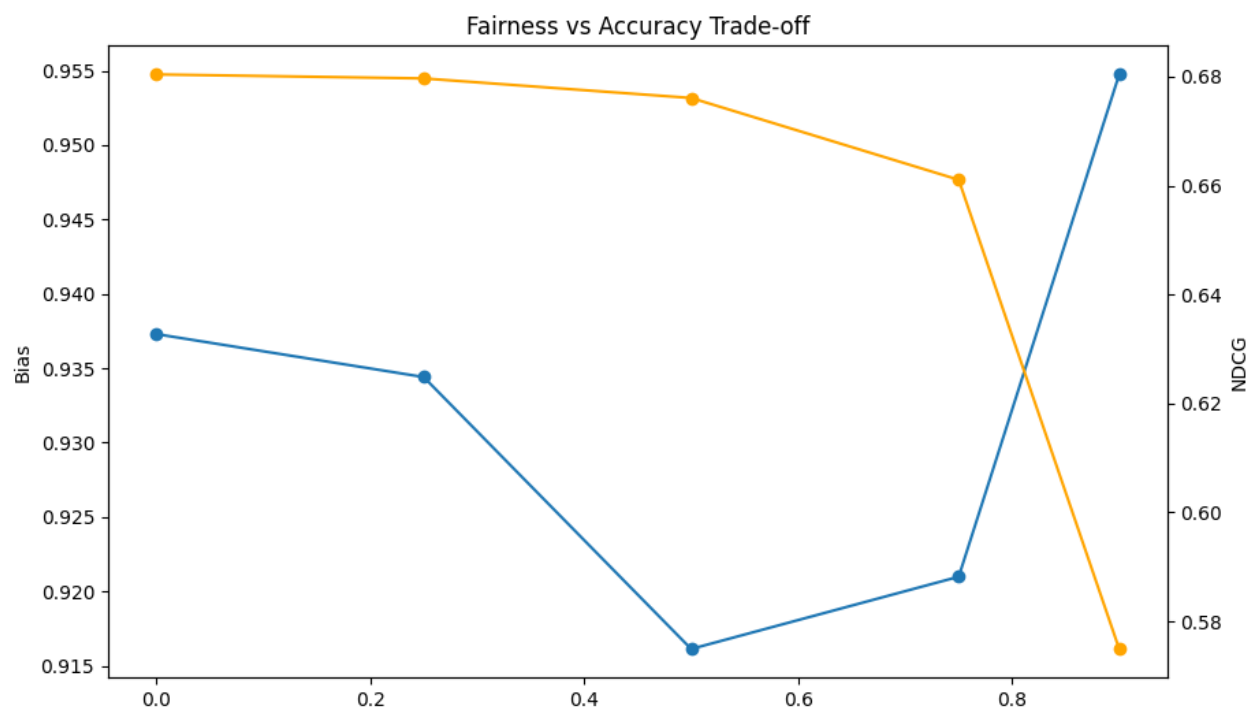


Figure 18: Important trade-off between fairness and accuracy. Moderate β values reduce bias while preserving recommendation quality, but very high β values decrease HitRate and NDCG.

13. Evaluation

This part of the report talks about the evaluation of the fairness-aware recommendation system that was put in place. We looked at this system in the way as we did in the other parts of the report. First we looked at how good the recommendations were. Then we looked at how fair the system was. Finally we compared how long it took to run.

We did these tests on a dataset that's similar to MovieLens, which has information about how users rated movies. This dataset also has information about the type of movie and the users themselves like their gender, age and what they do for work. This information helps us see if the recommendations are good and if they are fair, for groups of users.

We did all of these tests using Python in a Jupyter Notebook. We used a main tools to help us: pandas to work with the data NumPy to do math scikit-learn to help us split the data and evaluate it and matplotlib to make pictures of the data.

We used four fairness-aware recommendation system metrics to evaluate the fairness-aware recommendation system:

Metric	Purpose
HitRate	Measures whether recommendations include relevant movies
NDCG	Measures ranking quality
KL-Divergence	Measures fairness bias
Runtime	Measures computational efficiency

The final comparison of the main methods is shown below:

Method	HitRate	NDCG	Bias	Runtime
Baseline	0.957	0.680	0.937	~0
Fair Paper Method	0.955	0.676	0.916	67.94 s
Adaptive Method	0.958	0.680	0.899	94.46 s
Optimized Fair	0.955	0.676	0.916	1.24 s

The results show that the **baseline model** achieves strong recommendation accuracy, but it also has the highest bias. The **Fair Paper Method** reduces bias while keeping HitRate and NDCG almost unchanged, showing that fairness-aware re-ranking can improve fairness without strongly damaging recommendation quality.

The **Adaptive Method** achieves the best fairness score, reducing bias to **0.899**, while preserving almost the same accuracy as the baseline. This makes it the strongest method in terms of fairness improvement.

The **Optimized Fair Method** achieves the same HitRate, NDCG, and bias as the Fair Paper Method, but with a much lower runtime. It reduces runtime from **67.94 seconds** to **1.24 seconds**, making the fair re-ranking process much more efficient.

Overall, the adaptive method gives the best fairness score, while the optimized fair method gives the best runtime. The paper-style method provides a balanced middle point between fairness and accuracy.

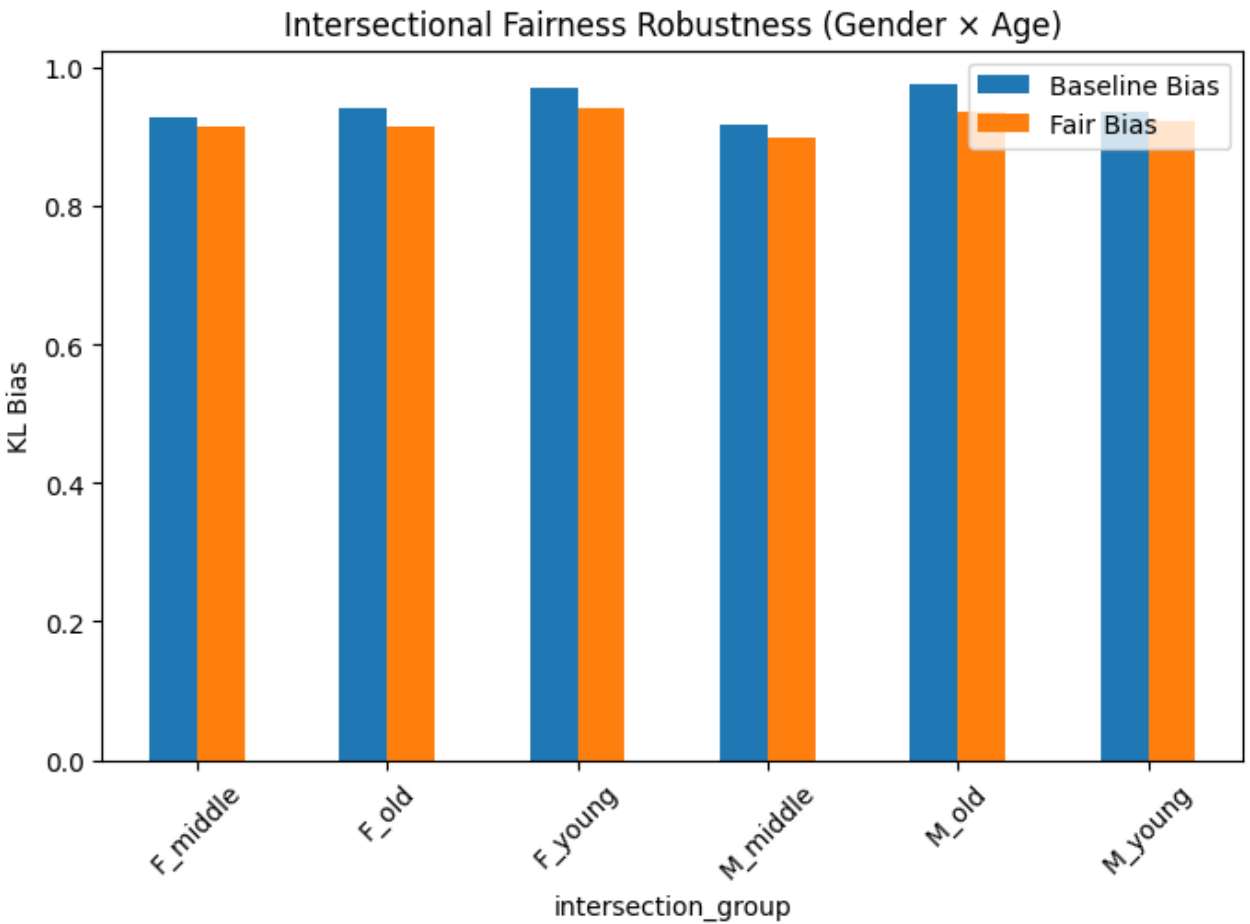


Figure 19: Final evaluation comparison of baseline, fair paper method, adaptive method, and optimized fair method across accuracy, fairness, and runtime metrics.

14. Conclusion & Future Work

In this project, we implemented a fairness-aware movie recommender system and evaluated how fairness re-ranking affects recommendation quality. The baseline recommender achieved strong accuracy, with high HitRate and NDCG values, but it also showed measurable demographic bias in the distribution of recommended movie genres.

To address this issue, we applied fairness-aware re-ranking based on both relevance and fairness. The results showed that the fair re-ranking method reduced KL-divergence bias while keeping recommendation quality almost unchanged. This confirms that fairness can be improved without significantly sacrificing accuracy.

We also extended the original approach by implementing an adaptive fairness method. Instead of using the same fairness weight for all users, the adaptive method assigns user-specific fairness weights based on each user's measured bias. This achieved the best fairness result in the project while preserving recommendation accuracy.

In addition, we optimized the fair re-ranking implementation. The optimized method produced the same HitRate, NDCG, and bias as the original fair method, but reduced runtime dramatically. This shows that fairness-aware recommendation methods can become more practical and scalable with efficient implementation.

Finally, robustness experiments showed that the method remains useful across different user groups, recommendation list sizes, and demographic settings. However, cold-start users still showed higher bias, indicating that fairness remains more difficult when limited user history is available.

Overall, the project demonstrates that fairness-aware re-ranking can reduce demographic bias in recommendation systems while maintaining strong recommendation performance. The adaptive and optimized versions further show that fairness can be improved both in effectiveness and efficiency.

Future Work

Future improvements could include:

- Improving fairness for cold-start users with very few interactions.
- Testing more protected attributes beyond gender, age, and occupation.
- Applying the method to larger datasets such as MovieLens 1M.
- Adding more advanced fairness metrics.
- Comparing the method with neural recommender models such as NeuMF or VAE-CF.
- Evaluating user satisfaction directly through user studies.
- Studying how fairness changes over time in dynamic recommendation systems.

References

[1]:

https://www.google.com/search?sca_esv=c002384ed01e5123&rlz=1C1GCEA_enGR1136GR1136&sxsrf=ANbL-n7ww0z9-_KSuZ49-s1QqwqzAUBdBg:1779280913894&udm=2&fbs=ADc_l-acTh1XOpH3U7blxRVfQR5uwuYj4r9gy2Evo0qcadc4dYDkhazAPtodrn_bK51i8Y7EC5al0ML9l7g2bNsr7sxGhBjXBZVkh_olljcdf5JAAmrbQlExY9L-PliYAjuz1hLDHSlN83QSjPQeG3Wvdpd3AF5b6EyVa5QVr0n-MgF4HlSnlRJbUk4rNiLx0b3t-nv8m1jJDz_Sid6WndCJlwZ2jsQTAFSSOeGDTuLShs0DDIZel8lk&q=recommendation+systems&sa=X&sqi=2&ved=2ahUKEwIU6M_28ceUAXUdJBAIHexJlpsQtKgLegQIKhAB&biw=1920&bih=911&dpr=1#sv=CAMSVhoyKhBILWJLLTlSZnd3cFI2SnhNMg5iSy05UmZ3d3BSNkp4TToOR0VFdktdERjTTJ0NU0gBCocCgZtb3NhaWMSEGUtYkstOVJmd3dwUjZKeE0YADABGAcgx6aA8AVKCBABGAEGASgB

[2]: <https://arxiv.org/abs/2506.18327>